
Source Auditing Lab Material

Course: *Introduction to Vulnerability Research Lab*

Author name: *Dr. Bernard Lampe*

Due date: *October 17th, 2019*

Introduction - Webservers

1. Popular webserver implementations are Apache, Microsoft's IIS and Nginx. These are mature and quite complicated programs.
2. This lab material is focused on AppWeb, which is an HTTP server supporting many embedded devices.
3. Appweb is specifically designed to be incorporated into small IoT or lights out (LO) management devices.
4. In addition to HTTP, web servers are also augmented with server side scripting languages to include Perl, Python, Ruby, Java, PHP, ASP and many others.

HTTP Overview

1. Hypertext Transfer Protocol (HTTP) is a text-based (versions 0.9, 1.0, 1.1) and binary-based (versions 2.0 and 3.0) protocol that is used to request resources from a server.
2. HTTP/1.1 was first documented in RFC 2068 in 1997. That specification was obsoleted by RFC 2616 in 1999, which was likewise replaced by the RFC 7230 family of RFCs in 2014.
3. HTTP/2 is a more efficient expression of HTTP's semantics "on the wire", and was published in 2015; it is now supported by major web servers and browsers over Transport Layer Security (TLS) using an Application-Layer Protocol Negotiation (ALPN) extension where TLS 1.2 or newer is required.
4. HTTP/3 is the successor to HTTP/2, using UDP instead of TCP for the underlying transport protocol. Like HTTP/2, it does not obsolete previous major versions of the protocol. HTTP/3 support was added in Cloudflare, Google Chrome, and Mozilla Firefox on 26 September 2019.
5. HTTP supports the commands of GET, HEAD, POST, PUT, DELETE, TRACE, OPTIONS, CONNECT and PATCH.
6. WebDav augments these commands with many more commands such as COPY, LOCK, MKCOL, MOVE, PROPFIND, PROPPATCH, and UNLOCK.
7. Most implementations only support a subset of these commands.

Sample HTTP 1.1 GET Request

```
1 GET /index.html HTTP/1.1
2 User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
3 Host: www.example.com
4 Accept-Language: en-us
5 Accept-Encoding: gzip, deflate
6 Connection: Keep-Alive
```

Sample HTTP 1.1 POST Request

```
1 POST /cgi-bin/process.cgi HTTP/1.1
2 User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
3 Host: www.example.com
4 Content-Type: application/x-www-form-urlencoded
5 Content-Length: length
6 Accept-Language: en-us
7 Accept-Encoding: gzip, deflate
8 Connection: Keep-Alive
9
10 licenseID=string&content=string&/paramsXML=string
```

Indexing Source Projects

1. Install exuberant ctags
 - (a) On Debian, “sudo apt-get install exuberant-ctags”.
2. Run ctags -R .
3. A “tags” file should be placed in the current directory.

Start Auditing AppWeb

1. Begin by looking at the protocol. Here we see keywords such as “Connection:”, “Accept-Language:”, “Host”, “User-Agent:”. We want to look for those keywords in the source code to determine where the requests are parsed. Then we can audit the code around it. Find the file and functions that do header parsing.

Find Calling Functions and Structure Definitions

1. Once you have found the <src dir>/http/httpLib.c:parseHeaders() function, use ctags or cscope to find the functions which call this function.
2. Also find the HttpConn and HttpPacket class definitions.

Continue Auditing

1. Walk the call stack. What functions call `parseHeaders()`, `parseIncoming()`, and `httpProtocol()`?
2. What do these functions do? Why did the developer split the functionality up this way?
3. Audit the parsing of all headers.
4. What HTTP commands does the server implement?
5. Milestone: find at least one real bug in AppWeb's header handling. Write a one-paragraph bug report (description, minimized reproducer input, exploitability statement per the lecture's **From Bug to Report** section) as your deliverable for this lab.

Heap Bug Lab: the `str_store` Program

1. The repository ships `heap.c`, a small string-store program with CREATE/DELETE/FIND commands. It compiles warning-free:


```
1 clang -g -O0 -fsanitize=address heap.c -o heap_demo
```
2. Baseline run (verified): CREATE "FOO", FIND "FOO", DELETE "FOO" — all behave, exit 0. The bug needs a second DELETE.
3. **Exercise A — trace the dangling pointer.** Modify `main()` to: CREATE "A", CREATE "B", DELETE "A", DELETE "A". Rebuild and run. Observed behavior under ASan (verified on this material's reference build):

```
1 ==pid==ERROR: AddressSanitizer: heap-use-after-free on address 0x...4d0
2 READ of size 1 at 0x... thread T0
3     #0 ... in strcmp
4     #1 ... in find      heap.c:14
5     #2 ... in str_store heap.c:31
6     #3 ... in main      heap.c:45
7 0x... is located 0 bytes inside of 256-byte region
8 freed by thread T0 here:
9     #1 ... in str_store heap.c:31 (the first DELETE)
```

Answer these before running:

- (a) Why does the second DELETE of "A" not report a *double-free* but a *use-after-free* in `strcmp`? (Hint: does DELETE remove the pointer from `g_strs`? What do subsequent command lookups do with the freed entry?)
- (b) Why is there no crash without ASan? What does the second `free(find(str))` do once ASan is out of the picture? (glibc/tcmalloc answer: double-free or corrupted freelist — explain what each allocator does.)
- (c) Reconcile with the lecture's UAF and Double Free sections: which bug class is this *in first manifestation*, and under which allocator is it the other class?

4. **Exercise B — fix the bug class, not the bug.** Propose the minimal structural fix that removes this entire class from the program (consider: NULL the slot on delete and have `find()` skip freed slots; discuss why patching only the one call site does not fix DELETE of “B” after “A” leaks). Rebuild, rerun your four-command sequence, confirm clean.
5. **Exercise C — exploitability assessment.** Without ASan, exercise the full command surface (many CREATEs, interleaved DELETEs, FINDs) and assess the resulting primitive against the lecture’s **Exploitation Mitigations** map: what would you need to groom (which allocation size class), what metadata lives in a same-size object in a real target, and which of the “ideal heap vulnerability qualities” are satisfied?
6. **Exercise C2 — allocator internals.** STRLEN_MAX is 0x100: a tcache-class size in glibc. On a Linux box (or under a glibc-accurate simulator):
 - (a) Instrument CREATE/DELETE with an `malloc_usable_size` printout and map every string to its chunk size class (0x100 payload → 0x110 chunk). Explain what lives in the first 8 bytes of a *freed* chunk of this class (tcache fd / next pointer) and connect it to what a debugger shows watching `g_strs[0]` flip to “\x90\x11” after free.
 - (b) State the post-free double-serve: with the block in tcache twice (the double-DELETE), the next two CREATEs return the same pointer. Groom check: CREATE “A”, CREATE “B”, DELETE “A”, DELETE “A”, then CREATE “C”, CREATE “D” — under plain glibc print the three pointers of A’s slot history. Do C and D alias A’s chunk?
 - (c) Now suppose a real target stores a function pointer in a 0x110-chunk object. Write the grooming sequence (in command form) that turns this program’s UAF primitive into an overwrite of that pointer, and say which mitigation row (per the map) governs this attempt.

Explore Other Tools

1. Download another version of Appweb (<https://www.embedthis.com/>) and use `meld` to compare files across the versions.
2. Install Eclipse with the C++ extension and index the project.
3. Install and use `cscope` with `vim`.